ENPM691 Homework 04: Program Memory Layout and Variable Storage

Nelay Sharma
University of Maryland
UID: 122295414
10/7/2025

***Abstract*—This homework is how C variables occupy memory across different memory segments in a 32-bit Linux process, and how compilers assign them to different segments such as the stack, heap, global (.data/,bss) areas. Using `gcc`, `GBD`, and standard binary analysis tools (e.g., `readelf`, `nm`, `objdump`), the homework visualizes where each type of variable resides in the process's virtual address space, and how the assembly code references it. The homework provides a understanding of how the C language interacts with system memory architecture at a low level, an essential for secure programming and memory-safe development practices. In short, this homework confirms stack variables have temporary lifetimes within function frames, heap variables persist until freed, and global/static variables occupy fixed locations in memory throughout program executions.**

I. Introduction

The purpose of this homework is to investigate the memory organization of a C program and understand how variable types influence where data resides in the process' address space. For instance, compilers assign local variables to the stack, allocate global and static variables in fixed segments, and use runtime library calls to manage heap memory dynamically. Understanding these behaviors is essential for vulnerabilities as it helps developers and security engineers see why certain bugs (i.e., buffer overflows, memory leaks, dangling pointers, use-after-free errors) happen and how debuggers like `GDB`, pwndbg, `objdump`, and `readelf` can help reveal these vulnerabilities. [1]

II. Methodology

A. Environment

All work was done in a Kali Linux virtual machine inside VMware Workstation Pro 17, with 80 GB of storage and 8 GB of memory. Compilation was performed with GCC 14.2.0 (Debian 14.2.0-1) in 32-bit mode using the `-m32` flag, and debugging/analysis was carried out with GDB 16.3. The architecture used was IA-32 (x86). Multilib support was installed to ensure successful `-m32` compilation. A dedicated directory named `hw4` (inside the Kali VM) was created to hold source code, compiled binaries, and analysis outputs. [2]

B. Programs

`layout.c`, declares a mix of global, local, and heap allocation variables through `malloc()`, then prints their memory addresses to show how they are distributed through memory. The flag -O0 disables optimization, `-fno-omit-frame-pointer` ensures the stack frame references are explicitly preserved, and `-no-pie` disables ASLR-based relocation. [5]

```
gcc -m32 -g -O0 -fno-omit-frame-pointer
-no-pie layout.c -o layout32
```

C. Artifacts

Artifacts collected for this assignment included:

- **Code:** The source file `layout.c`, demonstrating stack `(local)`, heap `(dynamic)`, and global `(.data)` variable allocation and address printing.
- **Structure:** The observed memory layout showing how variables are distributed across the stack, heap, and data regions.
- **Compiler Flags:** The build used `-m32 -g -O0 -fno-omit-frame-pointer -no-pie` to include debug symbols, disable optimizations, preserve stack frame references, and ensure consistent addressing.
- **Debugger Output:** A GDB session transcript (`gdb_output.txt`) showing breakpoints, variable addresses, and disassembly of `main`.
- **Disassembly:** Output from `objdump` (`disasm_objdump.txt`) highlighting stack-relative and absolute memory references.
- **Section/Symbol Data:** Information from `readelf` and `nm` mapping program symbols to ELF sections (.text, .data, .bss).
- **Program Output:** Raw runtime addresses (`runtime_stdout.txt`) verifying memory placement and variable segmentation.

III. Results

A. Program Execution

TABLE I
Observed Memory Layout of Variables at Runtime

Variable	Address (32-bit)	Memory Region
Local Variable	`0xffd6de34`	Stack (temporary)
Heap Allocation 1	`0x0976e1a0`	Heap (dynamic)
Heap Allocation 2	`0x0976e1b0`	Heap (dynamic)
Global Variable	`0x804c018`	`.data` (static/global)

The local variable found near `0xffd6de34` resides on the stack which is a temporary memory region tied to the function's stack frame. In addition, the other two heap allocations at `0x0976e1a0` and `0x0976e1b0` originate from the dynamically managed memory obtained from `malloc()`, which assigns space at runtime and persists until **explicitly** freed from the heap. Finally, the global variable at `0x804c018` resides in a fixed location in the `.data section` of the executable, which represents the statically allocated storage that remains constant throughout the program's execution and function calls. [4] [6]

B. Assembly Analysis

Disassembly via objdump confirmed this organization:

TABLE II
KEY ASSEMBLY INSTRUCTIONS (COMPACT)

Instruction	Operation	Region
`mov local, 5`	Initialize local variable	Stack
`call malloc`	Allocate dynamic memory	Heap
`push &g`	Load address of global `g`	.data

Legend: `local` ≡ `[ebp-0x14]`, `&g` ≡ `0x804c018`.

Here, the `%ebp` register references the stack frame, and the instruction `push $0x804c018` directly loads the address of the global variable `g` from the `.data` segment. [3] [6]

TABLE III
RELEVANT ELF SECTIONS AND THEIR MEMORY ADDRESSES

Section	Address	Description
`.text`	`0x08049070`	Executable instructions
`.data`	`0x0804c010`	Initialized global/static variables
`.bss`	`0x0804c01c`	Uninitialized global/static variables

In addition, `readelf` further confirmed the printed global addresses, which align with the observed value `0x804c018` and proves the global variable `g` is stored in the `.data` section of the executable. The output shown in Table III illustrates how each section is mapped in memory. [4]

C. Debugger Output

The automated `GDB` session recorded the memory addresses of each variable type during execution across the stack, heap, global regions. This directly correlates with the runtime results, the instruction `movl $0x5, -0x14(%ebp)` initializes the local variable on the stack, while `push $0x804c018` shows the global variable's address is an absolute reference within the `.data` segment. In contrast, heap variables exist only as dynamically allocated pointers created at runtime and referenced indirectly through their assigned memory addresses. [1] [4]

TABLE IV
DEBUGGER OBSERVATIONS OF VARIABLE ADDRESSES

Variable	Observed Value	Region
`local_var`	`0xffffcf44`	Stack
`heap1`	`0x0976e1a0`	Heap
`heap2`	`0x0976e1b0`	Heap
`g (global)`	`0x804c018`	.data

IV. DISCUSSION AND CONCLUSION

This homework confirms the canonical memory layout used by modern 32-bit Linux processes, where automatic local variables are stored on the stack, heap allocations are created dynamically at runtime, and static/global variables occupy fixed positions in the executable's data segments; With the compiler emitting frame-relative instructions for stack variables and absolute addressing for globals, allowing GDB and similar tools to clearly reveal each variable's storage location and lifetime.

From a debugging and security standpoint, understanding these relationships and behaviors is crucial. Stack overflows typically overwrite local data and control information, heap overflows corrupt adjacent dynamic objects, and global variables persist throughout execution. Observing assembly-level details such as `movl $0x5, -0x14(%ebp)` and `push $0x804c018`, we can see how the compiler translates high-level variable declarations into low-level memory references. Even in small programs, a variable's storage class directly impacts its lifetime, scope, and addressing.

In conclusion, this homework reinforces a fundamental principle of systems programming and computer security, where memory layout and segment behavior is essential for writing reliable, maintainable, and secure software.

ACKNOWLEDGEMENTS

Special thanks to Dr. Lewis Collier and TA Ilya Yatsenko for assistance in guidance for this homework assignment.

GLOSSARY OF TERMS

Stack Frame: A temporary memory region created per function call to hold local variables, saved registers, and the return address. It is automatically released when the function exits.

Heap Variables: Memory blocks dynamically allocated at runtime with `malloc`, until **explicitly** freed by the program.

REFERENCES

[1] ENPM691 Lecture 04, "Machine Level Programming 2," 2025.
[2] ENPM691 Course Materials, "programs_export.zip," 2025.
[3] Linux Foundation, "Special Sections," in *Linux Standard Base Core Specification, Version 1.1*, 2001.
[4] M. Kerrisk, "elf(5) — Executable and Linkable Format (ELF) file format," *Linux Man Pages*, man7.org, 2024.
[5] Free Software Foundation, "Frame Layout," in *GCC Internals*, GNU Project, 2024. [Online].
[6] OSDev Wiki, "ELF," *OSDev.org*, 2024. [Online].
[7] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

APPENDIX A
MAIN SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>

int g = 10;

int main(void) {
    int local = 5;
    int *heap1  = malloc(sizeof(int));
    int *heap2  = malloc(sizeof(int));

    *heap1 = 99;
    *heap2 = 123;

    printf("Address of local  : %p\n", (void*)&local);
    printf("Address of heap1  : %p\n", (void*)heap1);
    printf("Address of heap2  : %p\n", (void*)heap2);
    printf("Address of global : %p\n", (void*)&g);

    free(heap2);
    free(heap1);
    return 0;
}
```

APPENDIX B
BUILD AND EXECUTION COMMANDS

```
$ gcc -m32 -g -O0 -fno-omit-frame-pointer -no-pie layout.c -o layout32
$ ./layout32 | tee runtime_stdout_32.txt
```

APPENDIX C
DEBUGGER SESSION (GDB)

```
Breakpoint 1, main () at layout.c:7
7           int local = 5;
$1 = (int *) 0xffffcf44   ← local variable
$2 = (int *) 0x0976e1a0   ← heap1
$3 = (int *) 0x0976e1b0   ← heap2
$4 = (int *) 0x804c018 <g> ← global variable

Dump of assembler code for function main:
   0x08049197 <+17>: movl   $0x5, -0x14(%ebp)
   0x080491a3 <+29>: call   0x8049060 <malloc@plt>
   0x080491d7 <+81>: push   $0x804c018
```

APPENDIX D
RUNTIME OUTPUT

```
Address of local  : 0xffd6de34
Address of heap1  : 0x0976e1a0
Address of heap2  : 0x0976e1b0
Address of global : 0x804c018
```

Appendix E
Binary Inspection

```
$ readelf -S layout32 | egrep '\.text|\.data|\.bss'
  [12] .text  PROGBITS  08049070 ...
  [23] .data  PROGBITS  0804c010 ...
  [24] .bss   NOBITS    0804c01c ...

$ nm -n layout32 | egrep ' g '
0804c018 D g
```

Appendix F
Disassembly (Excerpt)

```
08049197: movl $0x5, -0x14(%ebp)   ; local variable (stack)
080491a3: call 0x8049060 <malloc@plt> ; heap allocation
080491d7: push $0x804c018          ; global variable (data)
```

Appendix G
Memory Map Snapshot (/proc)

```
56557000-56578000 rw-p 00000000 00:00 0  [heap]
ffcf0000-ffd10000 rw-p 00000000 00:00 0  [stack]
... libc.so.6 mapped (r-xp / r--p / rw-p segments) ...
... ld-linux.so.2 loader mappings ...
```

Appendix H
Compiler and Debugger Flags

- `-m32`: Compile for 32-bit IA-32 architecture.
- `-g`: Include debugging information for `GDB`.
- `-O0`: Disable compiler optimizations.
- `-fno-omit-frame-pointer`: Preserve frame pointers for easier stack tracing.
- `-no-pie`: Disable Position Independent Executable (PIE) to stabilize addresses.